

FIBER

Refinement-Directed Mathematical Reasoning over Lean

Scoped properties, behavioral contracts, and certified computation

Prepared for Vladimir Reshetnikov
OpenAI assistant

6 September 2026

Abstract

A mathematical object rarely enters an argument with only a coarse carrier type. It is a prime above a bound, an integrable function under a particular measure, a map with a specified inverse, or a formal solution observed to a stated precision. Lean can already express these descriptions, but their evidence is often separated from the expressions to which it applies. The result is repeated reconstruction of side conditions, representation bridges, and local hypotheses.

FIBER is a proposed refinement-directed elaboration layer over Lean. It makes such descriptions part of source-level typing while translating them to ordinary dependent types and proof terms. Its central operation is not unrestricted automatic coercion: it is a proof-producing, scope-sensitive conversion from the information already known about an object to the exact contract needed by its next use. Behavioral refinements also flow backward into synthesis, so they can constrain incomplete candidates rather than merely rank complete inhabitants of a coarse type.

The design is grounded in both revised round-two syntheses, selected ProveIt developments, the Frey-curve arithmetic in the Anthropic FLT corpus, and Leant's current behavioral-filtering boundary. It gives typing and elaboration rules, a non-vacuous exact-quotient case study, a new-to-this-review probability example, a mathematical soundness proof for a sparse multivariate certificate checker, and an implementation and evaluation plan. A companion program executes 1,298 finite checks in 31 groups. Those checks are not a Lean formalization; neither a complete language implementation nor a build of the source repositories is claimed.

Recommendation. Extend what the *elaborator treats as a type*, not what the trusted kernel treats as definitional equality. Make refinements compositional, contextual, and usable by synthesis; retain Lean as the final proof checker.

Contents

1	The proposal in one argument	4
1.1	What changes from round two	4
1.2	The essential negative decision	4
1.3	What would count as success	5
2	Source review and evidentiary boundaries	5
2.1	The two reports are mutually revised, not independent final votes	5
2.2	ProveIt: three scales of proof work	5
2.3	The FLT corpus: arithmetic invariants and generated context management	6
2.4	Leant: the current boundary is more developed than a simple type search	6
3	Which type system should be extended?	7
3.1	Lean already has the required logical expressiveness	7
3.2	Three implementation choices	7
3.3	Expressive refinements do not imply complete inference	7
4	Related ideas and what is borrowed	8
5	A small semantic core	8
5.1	Carriers, refinements, and anchored evidence	8
5.2	Refinement weakening is a proof-producing operation	9
5.3	Functions and behavioral refinements	10
5.4	Data refinements are not all of one kind	10
5.5	Property families form an indexed logical interface	10
6	Typing and elaboration rules	11
6.1	Judgments and obligations	11
6.2	The essential rules	11
6.3	Backward propagation through contracts	12
6.4	A non-vacuous elaboration theorem	12
6.5	Statement fidelity is a separate obligation	13
7	Property inference without an instance explosion	13
7.1	Separate structure data from propositions about it	13
7.2	Rule registration	13
7.3	Demand-driven closure	14
7.4	Definitions carry interfaces, not hidden axioms	14
8	Locality, measures, and precision are part of the type	14
8.1	A fact has a telescope, not only a location label	15
8.2	Refinement does not force a partial-function default	15
8.3	Approximation demand flows backward	16

9	Exact quotients and the Frey-curve example	16
9.1	The source proof’s real content	16
9.2	A non-vacuous extracted contract	16
9.3	The refined quotient interface	16
9.4	Definition-first construction	17
9.5	An elaboration trace the author can inspect	18
10	A probability example beyond the shared algebraic controls	18
10.1	Objects and their scoped properties	18
10.2	A concise proof with an exact contract	18
10.3	An additional quantitative consequence	19
10.4	What the type layer contributes	20
11	Controls: local differentiation and binomial inversion	20
11.1	Local differentiation must stay local	20
11.2	Binomial inversion must stay uniform and module-valued	20
11.3	An already-short result should remain short	21
12	Behavioral refinements as synthesis specifications	21
12.1	What a useful behavioral type must say	21
12.2	Synthesize data and evidence together	21
12.3	Partial candidates and the limits of pruning	22
12.4	Relational specifications and observational summaries	22
12.5	A concrete Leant integration sequence	22
12.6	Result taxonomy	23
12.7	Tactic suggestion is a different target	23
13	A verified-CAS boundary with an explicit checker argument	23
13.1	Three execution routes, one specification	23
13.2	A concrete sparse multivariate certificate format	24
13.3	Ideal-combination checking	24
13.4	The original-expression bridge	25
13.5	What this checker does not decide	25
13.6	Residual jets fit the same contract discipline	25
14	The author-facing language and its document model	26
14.1	A small vocabulary, with Lean terms as an escape hatch	26
14.2	Three views of one accepted derivation	26
14.3	Named reasons constrain rule selection	27
14.4	Independently check every assertion	27
15	Implementation architecture and a realistic first release	27
15.1	Module responsibilities	27
15.2	Do not duplicate Lean’s workspace	28

15.3	Elaboration pipeline	28
15.4	A staged prototype that tests the new idea	28
15.5	Performance hazards worth addressing early	29
16	Trust, replay, and migration	29
16.1	Logical trust is not an execution-status flag	29
16.2	Operational capabilities should not become logical assumptions	29
16.3	Retained evidence and checked reuse	30
16.4	Toolchain upgrades and collaboration	30
17	Evaluation: isolate the effect of refinements	30
17.1	An ablation that charges infrastructure fairly	30
17.2	Tasks and leakage	30
17.3	What to measure	31
17.4	A targeted mutation register	31
17.5	Stopping and revision criteria	32
18	Answers to the third-round engineering questions	32
19	Conclusion	33
A	Reference encoding and artifact status	34
B	Reproducing the finite checks	34
C	Source identities and review map	35

1 The proposal in one argument

A mathematician says: “The numerator is divisible by four, so regard this coefficient as an integer and then extend scalars.” A conventional formal proof may instead unfold two separately defined curves, visit each coefficient, establish divisibility, invoke a cast-of-division theorem, normalize casts, and finish with structural extensionality. None of these steps is logically superfluous. Their repeated *authorship*, however, need not be necessary.

The proposed language treats a description such as “an integer quotient of this numerator by four” as a usable source-level type. It retains the quotient equation and the identities of the numerator and denominator. The next operation, scalar extension, requests precisely that equation. A registered theorem supplies the conversion. No heuristic is permitted to reinterpret integer division as field division, and no new kernel rule is needed.

The same architecture applies to less elementary examples. Differentiation requests equality in a neighborhood, not equality at a point. Dominated convergence requests a common integrable bound and convergence under a particular measure, not simply a collection of pointwise limits. A synthesizer asked for a sorting function needs a permutation-and-ordering contract, not merely the carrier type `List A -> List A`. The language should make these requirements part of ordinary elaboration, rather than recover them afresh in each tactic script.

1.1 What changes from round two

The two current syntheses agree on the acceptance boundary: a computation proposes a value together with evidence of a fixed relation to the original input. They also distinguish observations from exact presentations, retain scoped guards, reject unsound promotions between guarantees, and insist on a strong Lean baseline. These are adopted, not presented as inventions of this round [1, 2].

FIBER adds a more specific answer to the user’s type-system question. The additions are:

1. **A refinement elaborator with an explicit carrier-preservation invariant.** Properties participate in bidirectional typing, but neither a proof searcher nor a property rule may silently replace the underlying object or its structure dictionaries.
2. **A compositional contract calculus for mathematical uses.** Locality, measure, precision, coefficient domain, and chosen maps are indices in propositions. Their admissible changes are theorem applications, with the different variance of each coordinate made explicit.
3. **Behavior-directed synthesis before candidate completion.** Component contracts generate dependent verification conditions for holes and recursive calls. Leant’s existing candidate production and replay filtering remain useful, but are not mistaken for this stronger facility.
4. **Definition-level removal of repeated representation work.** An exact-quotient interface and a canonical base-change construction replace repeated coefficient-by-coefficient transport without erasing the arithmetic that justifies it.

These mechanisms are individually related to established work on refinement types, occurrence typing, and synthesis. The contribution claimed here is their concrete integration with the reviewed mathematical workflows and Lean’s proof boundary, not historical priority for refinement typing.

1.2 The essential negative decision

It would be tempting to let a CAS prove two types equivalent and then treat them as definitionally identical. FIBER does not do so. Polynomial equality, equality of real functions on a domain, equivalence of mathematical structures, and equal finite observations are different relations. They do not become one conversion rule because all are useful in mathematics.

In particular, a proof of $n = m$ may transport a vector of length n to length m , but that transport is represented by a Lean term. An algebraic isomorphism may transport a theorem through a registered construction, but is not silently an equality of carriers. A finite jet may answer a finite coefficient request, but is not a replacement for its infinite semantic object. This retains the strongest part of the second-round discipline while allowing much more aggressive source-level inference.

1.3 What would count as success

The target is a reduction in repeated decisions and repairs by an author who already understands the mathematics. It is not a percentage reduction inferred from lexical tactic counts. A successful result might be an ordinary Lean library plus a refinement elaborator and an improved document view. A new standalone surface should be retained only if it improves authoring or reading beyond that equally equipped Lean environment.

The question is therefore not “Can this theorem be written in five attractive lines?” It is “Can those five lines be elaborated with predictable obligations, preserve the theorem’s actual hypotheses, explain failure at the right mathematical level, and survive changes to the surrounding library?”

2 Source review and evidentiary boundaries

2.1 The two reports are mutually revised, not independent final votes

This review uses Brainstorm commit `b052ec01`, with full commit identities in Appendix C. The first report is *Mathematical Objects, Certified Computation*; the second is the revised *Nine Workbenches*. The latter already incorporates the former’s corrections to the complete-solution bridge, neighborhood equality, collapsed enclosures, benchmark interpretation, and mutation counts. Repeating those as unresolved defects in the current second report would misread the source [1, 2].

Several particularly useful decisions survive their exchange. A request must preserve its originally quantified object. A checker correctness theorem does not certify an arbitrary native execution. An unchanged printed formula does not establish unchanged instance resolution. Search failure is not logical negation. Existing contexts, local instances, and theorem packages must be accounted for before a new “workspace” claims credit. All of these are constraints on FIBER.

The reports’ 51 canonical negative-test families are an editorial consolidation, not 51 executed tests. Likewise, their historical Lean experiments are evidence about their stated environments and workloads, not measurements performed for this article. Their polynomial timing family is not a benchmark of general sparse multivariate algebra. This article uses those findings to choose an architecture, not to assert performance figures for it.

2.2 ProveIt: three scales of proof work

The inspected ProveIt revision is `24ce8bd7`, which pins Lean 4.32.0. Three developments are especially informative [3, 4, 5, 6].

Binomial inversion. `BinomialInversion.lean` separates integer kernel orthogonality from the transformation theorem. Its general result needs only an additive commutative group for the values of a sequence. Interval reindexing, natural subtraction, integer casts, and scalar actions form a substantial part of the visible proof. A frontend that silently assumes a field would make this example shorter by proving a weaker theorem. FIBER must instead preserve the integer-kernel action and the uniform quantifier over sequence indices.

Autonomous iterated derivatives. `AutonomousIteratedDeriv.lean` is already a well-factored library theorem. Its induction step constructs an eventual equality from an identity on an open set and then applies the chain rule. Its hypotheses on an auxiliary derivative family are needed only at values in the range of the solution. This is a control against both overclaiming library novelty and inferring unnecessarily global smoothness.

Weak convergence of spline laws. `UniformSplineWeakConvergence.lean` supplies a different development example from those emphasized by the syntheses. It proves convergence of characteristic functions by dominated convergence, handles a shrinking translation, and invokes Lévy’s continuity theorem. Its source exposes measurable maps, pushforward integration, almost-everywhere statements, a probability-measure instance, and topology on bundled probability measures. This file is development material in the present design, not a sealed evaluation task. No exhaustive claim is made that it was absent from every earlier individual proposal.

2.3 The FLT corpus: arithmetic invariants and generated context management

The inspected Anthropic repository revision is `aa2d8b34`. Its README describes a Lean 4.33.1/Mathlib 4.33.0 environment and validation procedures. Those repository reports were read, but the full development and its independent-checking claims were not reproduced here [7].

The definition of `FreyPackage` is already a dependent record of data and mathematical facts: nonzero integers a, b, c , a prime exponent $p \geq 5$, the Fermat equation, a coprimality condition, and congruences modulo four and two. It is direct evidence that “put properties in types” is not by itself a new design. The interesting question is how properties of that record are consumed automatically without turning all of them into a global instance-search problem [9].

The file `S_FreyPackage_freyCurveInt_map.lean` proves that the integral model maps to the rational model. Its first substantive proof establishes divisibility of two numerators before invoking `Rat.intCast_div`. Further lemmas transport discriminants and c_4 and introduce a local `Fact q.Prime` for valuation reasoning. These are concrete examples of value-level invariants, representation theorems, and temporary instance bridges [10].

The inspected no-Frey-package module also has extensive generated instance and simplification-attribute exclusions, while the final solution itself is a short application of existing theorems. The exclusions demonstrate a real source-management burden in this snapshot; they do not, by themselves, establish a general performance pathology in Lean. A property layer that indiscriminately registers every implication as a typeclass instance risks reproducing rather than solving such a burden [11].

The repository’s proof-path document is unusually valuable because it states the exact strengths of the named results. For example, its level-lowering and modularity interfaces are tailored statements, not arbitrary versions of every theorem bearing those names. FIBER’s method labels must attach to such exact theorem types, not to informal labels like “Wiles” or “Ribet” [8].

2.4 Leant: the current boundary is more developed than a simple type search

The Leant revision is `3a40904b`. Its engine exposes candidate groups, rendered variants, and retained typed provenance. The verification module deliberately defines an opaque callback-acceptance receipt whose documentation says it is not itself a kernel proof or behavioral certificate [12, 13].

The current Length facility is an opt-in behavioral stage after carrier-type verification. It can rank candidates and, with explicit authority, omit candidates refuted by independently replayed counterexamples. It supports a bounded continuation through two candidate batches and a command-local counterexample bank. Raw solver statuses do not authorize the stronger conclusion. Its own documentation expressly leaves typed-prefix pruning,

counterexample-directed engine synthesis, further behavioral domains, and Lean-checked behavioral artifacts as proposed work [14].

This distinction determines the integration plan: preserve Leant’s existing negative-evidence discipline, but add source-level refinements that can constrain candidate construction and ultimately produce proofs of universal contracts in Lean. Do not relabel existing replay filtering as a completed refinement-type synthesizer.

3 Which type system should be extended?

3.1 Lean already has the required logical expressiveness

The coarse type $\text{List } A \rightarrow \text{List } A$ does not specify sorting. But

$$\{f : \text{List } A \rightarrow \text{List } A \mid \forall xs, \text{Sorted}(f(xs)) \wedge \text{Perm}(xs, f(xs))\}$$

is a type expressing a universal behavioral specification. No extension of Lean’s foundational logic is required to write it [15]. Subtypes already package a value and a proposition about that value; coercions already allow use of a subtype’s carrier where a less informative type is expected [16, 17].

What is missing from the proposed workflow is not the ability to state that type. It is systematic elaboration support for discovering, retaining, specializing, and composing the relevant proofs while keeping the underlying terms convenient to use. A programmer can manually achieve much of this with subtypes, structures, implicit arguments, tactics, and local facts. FIBER makes a particular combination into a predictable language service.

3.2 Three implementation choices

Choice	What it buys	Limitation or cost
Library and macros	Immediate compatibility; ordinary theorem packages; small experiments.	General expected-type-driven refinement inference and coordinated holes become awkward if scattered across macros.
Refinement elaborator	A coherent source typing judgment, generated verification conditions, source-to-evidence mapping, and synthesis feedback.	More elaboration machinery and debugging surfaces; must preserve statement interpretation.
Kernel extension	Potentially more primitive conversion or subtype rules.	Enlarged trusted implementation and migration burden; solver-dependent conversion would be particularly problematic.

The recommendation is the middle option, initially implemented through Lean’s extensible elaboration infrastructure and ordinary libraries. This is a genuine extension of the *source language’s typing behavior*, even though its target logic is unchanged. A separate parser is optional.

A primitive kernel subtype is not ruled out in principle. It would need a concrete advantage over the existing encoding, precise interaction with reduction and dependent elimination, and an independently checked implementation story. No such advantage has been established by these examples. Adding theorem proving to definitional conversion merely to save explicit transports is not justified.

3.3 Expressive refinements do not imply complete inference

FIBER permits predicates in full Lean Prop. Therefore a general request to establish refinement subtyping includes arbitrary theorem proving. For example, checking whether the one-element type refined by $\top$ can be used at the same carrier refined by P requires establishing P .

The language consequently makes no claim of complete automatic subtyping or principal refinement inference. It uses a bounded proof-producing fragment, explicit theorem application, and residual proof obligations. Termination of the elaboration *attempt* is obtained by budgets; logical adequacy is obtained from the produced evidence. Failure to infer a refinement means that the current attempt did not justify the requested use, not that the mathematical proposition is false.

There is also no need to infer the “strongest property” of every term. Even when such a phrase can be given a semantic interpretation, it is not a practical interface. A term carries a finite set of relevant proved summaries. Consumers request additional summaries on demand.

4 Related ideas and what is borrowed

Refinement types and refinement-directed synthesis have a substantial history. Synquid uses polymorphic refinement types as synthesis specifications and decomposes them to constrain components. Its round-trip checking is an important precedent for rejecting an incomplete application before enumerating all completions. FIBER borrows that objective but does not assume that arbitrary Lean propositions fall in Synquid’s automated fragment [19, 20].

LiquidHaskell’s refinement reflection relates program definitions to logical reasoning about them. That is relevant to attaching behavioral summaries to executable components. FIBER requires those summaries to have Lean proofs at the accepted boundary; merely sending a reflected equation to an external solver is not that boundary [21].

F* combines dependent programming, refinements, and proof automation. Its account of total functions is particularly pertinent: a behavioral specification of a mathematical function must not be “proved” by a diverging implementation. Dijkstra monads provide a principled way to describe effectful code through weakest preconditions. FIBER borrows contract composition, but deliberately keeps provider execution status separate from mathematical truth [22, 23].

Occurrence typing, as documented for Typed Racket, refines knowledge in branches and handles immutable aliases. It also distinguishes a predicate with information on both outcomes from a one-sided predicate. FIBER makes the analogous distinction between a complete decision procedure and a one-sided certificate checker: failure of the latter cannot introduce a negative mathematical fact [24].

These precedents mean that refinement syntax, backward constraints, or branch-sensitive facts should not be advertised as new in isolation. The proposed research contribution is the combination of exact mathematical object identity, dependent scope, theorem-backed property conversion, measure and locality indices, and proof-producing synthesis within the reviewed Lean workflows. Whether that combination is useful remains an empirical question.

5 A small semantic core

5.1 Carriers, refinements, and anchored evidence

For an ordinary Lean type $A : \text{Type}_u$ and predicate $P : A \rightarrow \text{Prop}$, write

$$A\{P\} = \{x : A \mid P(x)\}.$$

Its reference interpretation is Lean’s `Subtype P`. A value is (a, p) with $a : A$ and $p : P(a)$. At this semantic level, there is nothing implicit.

FIBER also offers an unbundled local presentation:

```

let a : A := expression
know P(a) by reason
use a where A {P} is expected

```

The last line elaborates to the appropriate subtype constructor or ordinary proof argument. It does not change the binding of a . This presentation matters because replacing a heavily used local variable by a tower of subtype wrappers can make dependent expressions and existing APIs harder to use.

An *anchor* is an elaborated term together with its local context and meaning-bearing parameters. Operationally it has a stable identifier and a source location; logically the authority is the actual term. Two variables with the printed name x in sibling branches are not the same anchor. Two different names for the same local term may share evidence through definitional identity or a checked equality transport.

The property index stores entries of the form

$$(a, P, p : P(a), \Gamma_p, \text{dependencies}, \text{origin}).$$

Here Γ_p is the telescope under which the proof is valid. The index is a retrieval structure over existing proof evidence, not a second context that can assert facts by metadata.

5.2 Refinement weakening is a proof-producing operation

The basic same-carrier subtyping rule is

$$\frac{\Gamma \vdash h : \forall x : A, P(x) \rightarrow Q(x)}{\Gamma \vdash A\{P\} \preceq A\{Q\}}.$$

Its elaboration is

$$\text{weaken}_h(a, p) = (a, h(a)(p)).$$

The carrier value is definitionally unchanged in this encoding. For an already fixed anchor a , the weaker local premise $P(a) \rightarrow Q(a)$ suffices; a theorem uniform in every $x : A$ should not be demanded unnecessarily.

Conjunction gives an intersection-like source type:

$$A\{P\} \& A\{Q\} \equiv A\{\lambda x. P(x) \wedge Q(x)\}.$$

This is not an intersection of arbitrary unrelated Lean types. Both properties concern the same carrier value. A pair (a, p) and (b, q) cannot be combined into that refinement without first establishing the required identity between a and b .

Proposition 5.1 (Same-carrier coherence). *Suppose two refinement-conversion paths start with the same value, preserve its carrier at every step, and end in the same subtype $A\{Q\}$. Their results are equal as subtype values.*

Proof. The two final carrier projections are equal because each path preserves the original carrier. Subtype extensionality then gives equality of the final values. Different proofs of $Q(a)$ do not distinguish subtype values. This argument does not identify different carrier maps or different structure dictionaries. $\square$

This limited coherence theorem is useful: ordinary weakening paths need not be globally pinned merely because their proof terms differ. In contrast, a choice between two embeddings, two bases, or two algebra structures is data-bearing and remains explicit.

5.3 Functions and behavioral refinements

A pre/postcondition contract for a pure function $f : A \rightarrow B$ is

$$\text{Behaves}_{P,Q}(f) := \forall x : A, P(x) \rightarrow Q(x, f(x)),$$

where $Q : A \rightarrow B \rightarrow \text{Prop}$. The source type

$$(x : A\{P\}) \longrightarrow B\{\lambda y. Q(x, y)\}$$

can be implemented by a dependent function between subtypes, or by a carrier function together with the displayed behavioral proof. The compiler must specify which representation it uses. These presentations are closely related, but an arbitrary partial-domain dependent function does not automatically define a total carrier function outside its domain.

For nondependent function interfaces, ordinary contravariance and covariance have a proof-producing form. If $P' \Rightarrow P$ and $Q \Rightarrow Q'$, a function accepting P and producing Q can accept the stronger input requirement P' and be advertised with the weaker output guarantee Q' . For dependent outputs, the predicates must be instantiated at the same input and transported along any changed input representation. FIBER does not apply a shallow arrow-subtyping rule that forgets those dependencies.

A function-level refinement can also express a relational property involving more than one call:

$$\text{Monotone}(f), \quad \forall x, x', R(x, x') \rightarrow S(f(x), f(x')).$$

Such a property is not reducible to an output predicate at one arbitrary input without retaining the relational quantification. It belongs in the type of the function object, rather than in a misleading pointwise annotation.

5.4 Data refinements are not all of one kind

Consider three superficially similar descriptions:

$$\{d : \mathbb{Z} \mid d \neq 0\}, \quad \{d : \mathbb{Z} \mid \text{IsUnit}(d)\}, \quad \{u : \mathbb{Z}^\times \mid \text{True}\}.$$

The first two refine an integer by different propositions. The third contains a unit object with inverse data and equations. The integer 2 satisfies the first refinement but not the second. Even when a proposition says an inverse exists, constructing a chosen inverse is a separate operation whose computational and logical requirements must be represented.

Similarly, $\forall x, \exists y, R(x, y)$ is a proposition. A function producing a witness for every x is data. In classical Lean it may be obtained noncomputably by choice, with the relevant axiom dependency; it should not be presented as an executable algorithm merely because the source notation hides the extraction.

5.5 Property families form an indexed logical interface

For a fixed object a , properties are ordered by proved implication. A domain change, a context substitution, or a representation map changes which properties can be used. This is the useful meaning of the name FIBER: information is attached to a particular object in a particular context, not stored in a global bag of adjectives.

The implementation does not require a category-theoretic framework. It needs ordinary theorem constants for the allowed changes. For example,

$$V \subseteq U \implies \text{EqOn}_U(f, g) \rightarrow \text{EqOn}_V(f, g),$$

and

$$M \leq N \implies \text{JetEq}_N(F, G) \rightarrow \text{JetEq}_M(F, G)$$

are different theorem families, with different indices. Neither permits extending equality to a larger domain or increasing precision.

6 Typing and elaboration rules

6.1 Judgments and obligations

Let $\mathcal{E}$ be the Lean environment, Γ an ordered telescope of data and proof bindings, and Δ the searchable index of already justified facts. The synthesis judgment is

$$\mathcal{E}; \Gamma; \Delta \vdash e \Rightarrow A\{P\} \rightsquigarrow (t, p; \Omega),$$

where $t : A$, $p : P(t)$ may initially contain registered obligation metavariables, and Ω records their exact telescopes and targets. Checking uses an expected refined type:

$$\mathcal{E}; \Gamma; \Delta \vdash e \Leftarrow A\{Q\} \rightsquigarrow (t, q; \Omega).$$

An elaboration is *accepted* only when every required obligation is discharged and the resulting Lean declarations pass kernel checking and the selected axiom policy. The judgment with unresolved Ω describes a draft, not a theorem.

Obligations are not a set of unscoped propositions. A later obligation may depend on an earlier chosen witness, and its type must mention that exact witness. For example,

$$y : B(x), \quad h : R(x, y), \quad z : C(x, y), \quad k : S(x, y, z)$$

is an ordered dependent problem. Turning it into four independent solver queries would lose meaning.

6.2 The essential rules

Introduction of a refinement. If ordinary elaboration gives $t : A$, checking it against $A\{Q\}$ generates the proof obligation $Q(t)$. The target t is frozen before that proof search. A solver cannot replace t by a different term that makes the predicate easier.

Elimination and expected use. A refined value can be projected to its carrier. When an API needs a proof of $Q(t)$, the elaborator consults the property index and registered conversions. The inserted evidence is an ordinary argument. No additional assumption is added to the enclosing theorem’s statement.

Application. Suppose a selected component f has a proved contract

$$h_f : \forall x, P(x) \rightarrow Q(x, f(x)).$$

Applying f to t generates $P(t)$ if it is not already known and produces the summary $Q(t, f(t))$. The user may request a stronger postcondition $S(t, f(t))$; this generates a further implication or direct proof obligation. It does not mutate h_f or silently strengthen its contract.

Lambda and recursive definition. To check a function contract, introduce an arbitrary input and its permitted precondition, then check the body against the instantiated output refinement. A recursive call may use its induction hypothesis only under the termination or well-foundedness conditions accepted by Lean. A contract cannot justify its own implementation through an unguarded recursive call.

Case split. A branch receives the proof of its case condition in its local telescope. On merging branches, only facts justified in every relevant branch, or a correctly guarded disjunction of facts, are exported. A theorem under $a \neq 0$ must not leak into the $a = 0$ branch through a cache.

For executable branching on P , the source needs an appropriate decision procedure or a declared noncomputable classical construction. For proof-only case analysis, classical reasoning may be used under the project’s ordinary logical policy. These are distinct implementation routes.

Equality transport. From $h : a = b$ and $p : P(a)$, construct a proof of $P(b)$ with equality elimination. For a relation other than equality, use its registered respectfulness theorem. Equality on a set does not justify transport of a property that inspects values outside that set.

Requested computation. For a frozen request r with output type $O(r)$, a producer proposes $o : O(r)$. Acceptance requires a Lean term of $\text{Spec}(r, o)$ at the request’s telescope. A request for a conditional result and a request for an unconditional result have different specification types.

6.3 Backward propagation through contracts

Forward summary inference is not sufficient for synthesis. Suppose a hole h is used in the composition $g \circ h$, and the requested behavior is $S(x, g(h(x)))$. A contract $C_g(y, z)$ can induce the sufficient requirement

$$Q_h(x, y) := \forall z, C_g(y, z) \rightarrow S(x, z).$$

This is useful only if C_g is actually established for g at that input. The generated obligations include the component’s precondition as well. One sound composition theorem is

$$\begin{aligned} & (\forall x, P(x) \rightarrow Q(x, h(x))), \\ & (\forall x, y, P(x) \rightarrow Q(x, y) \rightarrow C_g(y, g(y))), \\ & (\forall x, y, z, P(x) \rightarrow Q(x, y) \rightarrow C_g(y, z) \rightarrow S(x, z)) \\ & \implies \forall x, P(x) \rightarrow S(x, g(h(x))). \end{aligned}$$

The proof is three applications of the supplied hypotheses. The practical difficulty is choosing a tractable Q and suitable component summaries, not this logical composition.

FIBER initially uses registered sufficient templates rather than automatically searching the full lattice of predicates. Failure of one decomposition only rejects that decomposition. A candidate might still satisfy the target through a different proof, so local inference failure must not be promoted to a universal exclusion of the candidate or all its completions.

6.4 A non-vacuous elaboration theorem

A soundness statement must say more than “the final term is checked.” Define the reference translation of a source refinement as a subtype; translate declarations in telescope order; translate each conversion to its supplied theorem application. Then require the following invariant for every elaboration rule.

Theorem 6.1 (Relative elaboration soundness). *Assume the source declarations have fixed interpretations in $\mathcal{E}$, every property-conversion rule has the stated Lean proof, and every obligation in Ω is discharged under its recorded telescope. An accepted derivation of the checking judgment for $e : A\{P\}$ produces a Lean term $t : A$ and a Lean proof $p : P(t)$ under the translated context. Same-carrier refinement steps preserve the selected carrier term. No fact with a free variable outside that context is required by the resulting proof.*

Proof. Induct on the elaboration derivation. Refinement introduction uses the supplied proof of $P(t)$. Weakening applies the registered implication to the induction hypothesis while retaining t . Application instantiates the component contract at the same argument term. Lambda introduction extends the telescope and abstracts the body proof on exit. Equality transport applies equality elimination to the induction hypothesis. A case split constructs the appropriate eliminator with separately checked branch proofs; the exported target has the outer telescope. A computation node composes its checker theorem and execution evidence with its original-target bridge. In each case, the constructor or theorem application has exactly the required Lean type. Telescope checking gives the free-variable condition. Carrier preservation follows syntactically in the same-carrier cases. $\square$

This is a proof about the specified calculus. It is not a machine-checked theorem about an implemented FIBER compiler. The kernel checks generated terms; a separate elaboration correctness and rendering discipline is still needed to connect those terms to the author’s intended source.

6.5 Statement fidelity is a separate obligation

Before searching for proofs, elaborate and record the theorem statement, including universes, coercions, notation, structure dictionaries, and quantifier dependencies. Distinguish an explicitly delegated construction hole from an unresolved statement hole. The former is allowed to select a witness satisfying a fixed specification; the latter must not be resolved by searching for the easiest theorem to prove.

The interface should show meaningful choices in an expanded statement view. A machine-checkable printer/parser round trip for the restricted source AST is desirable, but it cannot prove that a reader understood the intended mathematics. The blind reading experiment in Section 17 addresses that different question.

7 Property inference without an instance explosion

7.1 Separate structure data from propositions about it

A ring structure determines operations and is data-bearing. A proof that a particular element is nonzero is a proposition about those operations and that element. A continuity claim depends on selected topologies. An integrability claim depends on a measure. Even a proposition-valued class can be indexed by meaning-bearing data.

FIBER therefore resolves algebraic and analytic structures through Lean’s existing mechanisms, records the selected dictionaries, and only then infers propositions about the resulting expressions. It does not infer a new field structure merely because a tactic would benefit from one. Nor does it use proof irrelevance to identify different multiplication operations on the same carrier.

A local fact can be exposed to an existing API that expects `Fact P`, but this is an explicit, scoped bridge. The valuation proof in the Frey source is a useful model: a known primality fact is made available as a temporary instance at a specific point in the proof. FIBER can generate that bridge and record its extent instead of globally registering every derived proposition [10].

7.2 Rule registration

A property rule consists of an ordinary Lean theorem, a checked assignment of semantic roles to its binders, and non-logical search metadata. A typical rule has the form

$$\forall \vec{x}, H_1(\vec{x}) \rightarrow \cdots \rightarrow H_k(\vec{x}) \rightarrow Q(F(\vec{x})).$$

The metadata says which conclusion shape should index the rule, which arguments are inferred from the requested object, and what search budget is appropriate. It does not change the theorem’s premises.

Rules should be namespaced and versioned through project-owned wrappers. A renamed upstream binder should require fixing one wrapper, not silently assigning a semantic role to a different argument across many documents. Tests should deliberately change binder order, add a premise, and alter a coefficient domain to verify that registration validation notices the change.

7.3 Demand-driven closure

The first implementation should perform demand-driven backward search over a small collection of rules, with memoized proved results. Limited forward propagation is useful for cheap consequences such as positivity implying nonzeroness, or a conjunction exposing one conjunct. Unrestricted saturation over every imported theorem is neither needed nor desirable.

A cache key includes the elaborated target proposition, its actual local-variable identities, relevant environment dependencies, and the search profile. Hashes accelerate lookup; they are not logical evidence that two expressions have the same denotation. Evidence reuse across a changed context requires a checked substitution or implication bridge.

Cyclic rule graphs are not automatically unsound: $P \rightarrow Q$ and $Q \rightarrow P$ are perfectly legitimate theorems. But they cannot create either fact without a seed. The implementation must avoid looping, and a computation result must not be used to establish the prerequisite that authorizes that same result. The kernel’s finite proof term is the final protection; an acyclic obligation-assembly graph gives the author an earlier and more intelligible diagnostic.

7.4 Definitions carry interfaces, not hidden axioms

A declaration may ask the system to derive named interface facts:

```
def phase(t : Real, x : Real) : Complex := exp(I * t * x)
  derives continuous_in_x
  derives norm_is_one
```

This is proposed syntax. Each `derives` clause creates a theorem obligation with an ordinary Lean target. If it remains open, the definition can exist but the claimed interface cannot be published as proved. An imported definition package should expose its actual theorem inventory and dependency cost.

The same interface mechanism supports exact quotients, normalized coefficient observations, measurable pushforwards, and finite representations of formal series. It is more useful to implement a few such packages with real proofs than to introduce a universal declaration language before their needs are understood.

8 Locality, measures, and precision are part of the type

A list of adjectives such as “smooth, positive, convergent” is too imprecise for mathematical elaboration. FIBER’s predicates carry the arguments that make those descriptions meaningful.

Information available	An admissible use	An inadmissible automatic promotion
$\forall x \in U, f(x) = g(x)$	Restrict to a proved subset of U .	Equality on all of $\mathbb{R}$.
U open, $x \in U$, equality on U	Equality eventually at $\mathcal{N}(x)$.	Equality near x from equality only at x .
$f = g$ almost everywhere for μ	Transport suitable μ -integral statements.	Pointwise equality or equality for an unrelated measure.
$\text{Integrable}(g, \mu)$	Supply the common bound to a μ -dominated-convergence theorem.	Integrability under a different measure.
$F \equiv G \pmod{X^N}$	Read a coefficient of index $k < N$.	Read coefficient N or infer full series equality.
$l \leq a \leq u$	Weaken to wider bounds; use a strictly positive lower bound.	An exact decimal for a nondegenerate interval.
A product identity for factors	Reconstruct the original polynomial.	Irreducibility or completeness of a factor list.

These distinctions are inherited from round two, but here they are ordinary expected-type mismatches with precise missing bridges. The improvement sought is a common elaboration mechanism rather than a separate special-purpose warning for each case.

8.1 A fact has a telescope, not only a location label

Suppose a branch introduces $x \in U$ and proves $f(x) \neq 0$. The proof can be exported as an implication or universally quantified theorem if the appropriate binders are abstracted. It cannot be exported as an unconditional fact about an unrelated x .

The same issue arises with a measure or a filter. A cache entry for $f = g$ almost everywhere under μ cannot answer a request under ν merely because both measures are called “volume” in different namespaces. A bridge may exist, for example from an absolute-continuity hypothesis, but the theorem and its direction must be explicit.

For countably many indices, $\forall n, \text{AE}_\mu x, P(n, x)$ can be combined into $\text{AE}_\mu x, \forall n, P(n, x)$ using the countable intersection of conull sets. There is no unrestricted analogous rule for an uncountable family. Thus a proof for each real parameter t must not silently become one conull set on which the statement holds simultaneously for every t .

8.2 Refinement does not force a partial-function default

Existing Lean mathematical operations, including division and derivatives, often have totalized definitions. A frontend that lifts existing files should preserve those semantics. FIBER therefore retains the underlying Lean interpretation by default and makes partial-function packages explicit.

For example, the total-real expression $(x^2 - 1)/(x - 1)$ takes value zero at $x = 1$, whereas $x + 1$ takes value two. An identity between them is guarded by $x \neq 1$. A partial expression with domain $x \neq 1$ is a different object whose value is defined only on that domain. Extending it to a total function creates a new object and requires an agreement theorem; simplification must not choose the extension silently.

A partial package can use

$$D : X \rightarrow \text{Prop}, \quad v : \prod_{x:X} D(x) \rightarrow Y.$$

Its user-visible domain is part of its interface. An exact equivalence of partial expressions preserves the relevant domain as well as values; agreement only on a smaller intersection is a weaker result. These distinctions should survive printing.

8.3 Approximation demand flows backward

A requested coefficient of order k requires a jet of precision greater than k . A requested derivative jet modulo X^N generally requires the input modulo X^{N+1} . A request for an inverse series requires invertibility of the constant coefficient, not merely its nonzeroness in an arbitrary ring.

These are good synthesis constraints because they can be propagated before the computation runs. The planner may request more information than strictly necessary, but never less and then label the result with the requested stronger precision. A retained high-precision result may be weakened through a theorem; precision belongs in its semantic specification, not merely its cache filename.

9 Exact quotients and the Frey-curve example

9.1 The source proof's real content

In the inspected definition, the integral and rational curves have coefficients

$$a_1 = 1, \quad a_3 = a_6 = 0, \quad a_2 = \frac{b^p - 1 - a^p}{4}, \quad a_4 = -\frac{a^p b^p}{16}.$$

The same printed slash denotes integer division in one definition and rational division in the other. The source map theorem correctly proves exact divisibility before identifying the two interpretations [9, 10].

The arithmetic is elementary but indispensable. If b is even and $p \geq 4$, then $16 \mid b^p$, hence $16 \mid -a^p b^p$. If $a \equiv 3 \pmod{4}$ and p is odd, then $a^p \equiv -1 \pmod{4}$. Also $4 \mid b^p$. Therefore

$$4 \mid b^p - 1 - a^p, \quad 16 \mid -a^p b^p. \quad (\text{F})$$

The Fermat equation, c , nonzeroness of a, b, c , and coprimality are not needed for this map theorem's arithmetic core.

9.2 A non-vacuous extracted contract

This observation matters for both API design and evaluation. A full Frey package is ultimately shown contradictory by the surrounding FLT development. In an unrestricted environment, a prover could establish any requested coefficient property from that contradiction. That would be logically valid and completely uninformative about whether exact-division elaboration works.

Use the following consistent microbenchmark instead:

$$a, b \in \mathbb{Z}, \quad p \in \mathbb{N}, \quad \text{Odd}(p), \quad 4 \leq p, \quad a \equiv 3 \pmod{4}, \quad 2 \mid b. \quad (\text{F}_0)$$

For example, $a = 3, b = 2, p = 5$ satisfies these conditions. Prove the curve transport under (F_0) , then obtain the original package theorem by a short adapter. This preserves the actual mathematical work and prevents a final FLT theorem from acting as an evaluation shortcut. No claim is made that every helper in the original file admits the same weakening.

9.3 The refined quotient interface

Define

$$\text{ExactQuotient}(n, d) := \{q : \mathbb{Z} \mid dq = n\}.$$

This type makes sense even for $d = 0$; it is empty when $n \neq 0$ and not uniquely inhabited when $n = 0$. The ordinary exact-division constructor deliberately uses the stronger interface

$$\text{exactDiv} : \prod_{n,d:\mathbb{Z}} (d \neq 0) \rightarrow (d \mid n) \rightarrow \text{ExactQuotient}(n, d).$$

It can retain Lean's integer quotient as its carrier and prove its balance equation. The nonzero guard makes the quotient unique, which is important when the interface is meant to denote the already chosen integer division operation rather than an arbitrary solution of $dq = n$.

Proposition 9.1 (Exact quotient transport). *Let $n, d, q \in \mathbb{Z}$ with $d \neq 0$ and $dq = n$. Then, under the standard embedding into $\mathbb{Q}$,*

$$\iota(q) = \frac{\iota(n)}{\iota(d)}.$$

More generally, any ring homomorphism $j : \mathbb{Z} \rightarrow R$ preserves the balance equation $j(d)j(q) = j(n)$. Solving that equation by an inverse requires that $j(d)$ be a unit.

Proof. Apply the homomorphism to $dq = n$ and use preservation of multiplication. In $\mathbb{Q}$, injectivity gives $\iota(d) \neq 0$, so multiplication by its inverse yields the displayed identity. In a general ring, only the balance equation follows without an invertibility hypothesis. If $u : R^\times$ has underlying value $j(d)$, multiply by u^{-1} to obtain the corresponding inverse formula. $\square$

This separates three capabilities often conflated by automated algebra: exact division in the integer source, preservation of its balance equation, and inversion of the denominator in the destination. A nonzero integer need not remain invertible after an arbitrary scalar extension.

9.4 Definition-first construction

A proposed source-level definition is:

```
def integralModel(a, b, p satisfying F0) : WeierstrassCurve Int :=
{ a1 := 1,
  a2 := exactdiv (b^p - 1 - a^p) by 4,
  a3 := 0,
  a4 := exactdiv (-(a^p) * b^p) by 16,
  a6 := 0 }

let rationalModel := base_change(Int_to_Rat, integralModel)
characterize rationalModel by the displayed rational coefficients
```

The two `exactdiv` expressions generate the concrete guards in (F), as well as the trivial nonzero-denominator facts. The quotient interface retains the proved balance equations. A general coefficientwise base-change theorem and Proposition 9.1 then establish the characterization.

For a new development this construction avoids independently defining two objects that immediately need to be shown equal. For the existing repository, do not replace the original rational curve without proof. Instead build a compatibility theorem equating the new base-change definition to the existing `freyCurve`, and use that theorem as the migration boundary.

9.5 An elaboration trace the author can inspect

Node	Requested fact	Mathematical evidence
Q1	$4 \mid b^p - 1 - a^p$	Evenness of b , lower bound on p , oddness of p , residue of a .
Q2	$16 \mid -a^p b^p$	$16 \mid b^p$, then divisibility of a product.
Q3	$4q_2 = b^p - 1 - a^p$	Exact-division constructor with Q1.
Q4	$16q_4 = -a^p b^p$	Exact-division constructor with Q2.
Q5	Rational formulas for $\iota(q_2), \iota(q_4)$	Quotient transport at the same numerators and denominators.
Q6	Equality of the curve records	Coefficientwise extensionality and the trivial remaining coefficients.

An author normally sees the divisibility argument and the characterization. A diagnostic can expand Q1 into its congruence and exponent obligations. The proof terms are retained regardless of visibility.

The missing-premise tests are sharp. Dropping oddness admits $a = 3, b = 2, p = 4$, for which the first numerator is not divisible by four. Dropping evenness of b admits $a = b = 3, p = 5$, again failing that divisibility. Dropping the lower exponent bound admits $a = 3, b = 2, p = 3$, for which the second numerator is not divisible by sixteen. The companion checks these neighbors and 288 non-vacuous positive parameter triples. The general arithmetic proof above, not that sample, establishes (F).

10 A probability example beyond the shared algebraic controls

10.1 Objects and their scoped properties

The inspected spline development uses a probability space of coordinate sequences $\omega_j \in [0, 1]$ and the partial sums

$$X_n(\omega) = \sum_{j < n} \frac{\omega_j}{2^{j+1}}, \quad X(\omega) = \sum_{j \geq 0} \frac{\omega_j}{2^{j+1}}.$$

The source proves convergence of the laws, after a midpoint translation, using characteristic functions and dominated convergence [6]. For the proof below, independence of coordinates is not needed: measurability, the coordinate bounds, and a probability measure μ suffice. The specific distributional identification in the repository uses its existing definitions.

A useful definition package records

$$\text{Measurable}(X_n), \quad \text{Measurable}(X), \quad \forall \omega, X_n(\omega) \rightarrow X(\omega), \quad \mu(\Omega) = 1.$$

For each fixed $t \in \mathbb{R}$, put $\phi_t(x) = e^{itx}$. A phase-function package provides continuity, measurability, and $|\phi_t(x)| = 1$. Composition then yields the measurable integrands required for

$$\int \phi_t(X_n) d\mu \longrightarrow \int \phi_t(X) d\mu.$$

The common majorant is the constant function one, integrable because μ is a probability measure. Each fact mentions the same μ , functions, and parameter t .

10.2 A concise proof with an exact contract

```

fix t : Real
let h(n, w) := phase(t, X(n,w))
let hlim(w) := phase(t, Xlim(w))

know h(n, -) is AE_strongly_measurable under mu
  by phase_measurability and measurable_X(n)
know norm(h(n,w)) <= 1 for every n,w by phase_norm
know h(n,w) tends_to hlim(w) for every w
  by continuity_of_phase and partial_sum_limit

show integral_mu(h(n,-)) tends_to integral_mu(hlim)
  by dominated_convergence with majorant 1

```

This is proposed syntax, not a claim that the listed phrases are existing tactics. The method wrapper's type is the precise dominated-convergence theorem appropriate to complex-valued integration. Its obligations include the common measure, measurability, integrability of the majorant, the norm bound, and the required convergence statement. The wrapper may derive almost-everywhere assertions from universal ones; it must not infer them from a finite sample of ω .

The measured baseline should receive the same phase and dominated-convergence wrappers. The question is whether the refinement elaborator usefully assembles them from the types of the local objects, compared with manually applying those wrappers in Lean.

10.3 An additional quantitative consequence

The geometry of the same source yields a simple stronger observation. It is proved here to exercise a quantitative behavioral contract, not claimed as a new result in probability theory.

Proposition 10.1 (Centered coupling and characteristic-function bound). *Let $0 \leq \omega_j \leq 1$, let $\delta_n = 2^{-n-1}$, and define $Y_n = X_n + \delta_n$. Then*

$$|Y_n(\omega) - X(\omega)| \leq \delta_n.$$

If these are measurable random variables under a probability measure μ , then for every real t ,

$$\left| \int e^{itY_n} d\mu - \int e^{itX} d\mu \right| \leq \min\{2, |t|\delta_n\}.$$

Consequently the characteristic functions converge uniformly on every bounded interval of frequencies.

Proof. The coordinate bounds give absolute convergence and

$$0 \leq X - X_n \leq \sum_{j=n}^{\infty} 2^{-j-1} = 2^{-n} = 2\delta_n.$$

Subtracting the midpoint δ_n proves the first bound. The elementary inequality $|e^{iu} - e^{iv}| \leq |u - v|$ follows by integrating the derivative of $s \mapsto e^{is}$ along the real interval between u and v ; that derivative has norm one. Apply it with $u = tY_n(\omega)$ and $v = tX(\omega)$, integrate the norm bound, and use $\mu(\Omega) = 1$. The same difference is pointwise bounded by two, which gives the other bound. For $|t| \leq T$, the result is bounded by $T\delta_n \rightarrow 0$ uniformly in t . $\square$

The source already distinguishes qualitative weak convergence from its separate quantitative CDF theorem. FIBER should preserve the same discipline. Proposition 10.1 gives a frequency-domain bound; it is not automatically the source's uniform CDF bound. A new metric or observation requires its own bridge theorem.

10.4 What the type layer contributes

This example tests more than tactic compression. The property layer must retain the measure attached to integrability, recognize the probability-measure capability needed for a constant bound, and compose a relational Lipschitz property with a quantitative coupling bound. It must also distinguish the already proved pointwise convergence from a requested uniform statement.

A missing probability or finite-measure assumption should produce an obligation about integrability of the proposed constant majorant. It should not quietly strengthen the ambient measure space. A change from μ to another measure invalidates the relevant summaries unless a transport theorem is supplied. These are ordinary typing obligations at a mathematical level rather than mysterious failures deep inside a Bochner-integral proof.

11 Controls: local differentiation and binomial inversion

11.1 Local differentiation must stay local

Use the actual generality of the ProveIt autonomous theorem. Let $U \subseteq \mathbb{R}$ be open, let $W' = \varphi \circ W$ on U , and suppose

$$\begin{aligned} G_0(W(x)) &= W(x), \\ \text{HasDerivAt}(G_n, G'_n(W(x)), W(x)) &\quad (x \in U), \\ G_{n+1}(W(x)) &= G'_n(W(x))\varphi(W(x)). \end{aligned}$$

Then $W^{(n)}(x) = G_n(W(x))$ for $x \in U$ [5].

The successor step should typecheck as follows. The induction hypothesis is an equality on U . Openness and $x \in U$ turn it into equality near x . A derivative-transfer theorem uses that neighborhood equality. The chain rule applies at $W(x)$, where the auxiliary derivative hypothesis is available. The recurrence identifies the result.

A proposed command “differentiate the previous identity on U ” is therefore a macro over four theorem applications and their exact premises. It is not differentiation of an isolated equality at x . Nor may it replace range-local hypotheses by differentiability of every G_n on all of $\mathbb{R}$. The existing theorem is already a good adapter; the new work is property inference and an inspectable derivation, not rediscovering the theorem.

11.2 Binomial inversion must stay uniform and module-valued

For an additive commutative group M , the source theorem has the form

$$\left(\forall n, b_n = \sum_{k=0}^n \binom{n}{k} \cdot a_k \right) \implies \forall n, a_n = \sum_{k=0}^n (-1)^{n-k} \binom{n}{k} \cdot b_k,$$

where the coefficients act through integer scalar multiplication [4]. The refinement is a property of the *pair of sequences*, not a property of their values at one fixed index.

The contract for the change of variable $k = j + i$ should expose a bijection between finite index sets, with bounds, coverage, and equality of summands. The polynomial and integer-arithmetic services can discharge coefficient identities, while the transformation theorem preserves the additive-group carrier. A generic rational-series implementation must not force M to become a vector space.

The current syntheses already give the counterexample showing that one forward row does not imply the corresponding inverse row. At $n = 1$, let $a_0 = a_1 = 0$, $b_0 = 1$, and $b_1 = 0$. The forward row holds, while the inverse row would require $0 = -1$. Therefore a compact renderer must preserve the uniform quantifier visibly or through an unambiguous enclosing declaration [1, 2].

11.3 An already-short result should remain short

The final weak-convergence theorem in the spline file is already a direct application of the probability-measure characteristic-function theorem. Likewise, several binomial corollaries are applications of the generalized transform interface. FIBER should accept an ordinary Lean term or tactic block unchanged, rather than force it into a longer sequence of narrative commands. Compression of long infrastructure proofs and preservation of short well-factored proofs are separate evaluation requirements.

12 Behavioral refinements as synthesis specifications

12.1 What a useful behavioral type must say

Consider synthesis of $f : \text{List } \mathbb{N} \rightarrow \text{List } \mathbb{N}$. A length refinement

$$\forall xs, |f(xs)| = |xs|$$

rejects some wrong candidates but admits reversal, the identity, arbitrary permutations, and a same-length list of zeros. Even sortedness together with length preservation admits the list of zeros. Equality of the sets of elements also loses multiplicity: $[1, 1, 2]$ and $[1, 2, 2]$ have the same underlying set.

For a linear order on A , a correct sorting contract is

$$\text{SortSpec}(f) := \forall xs, \text{Sorted}_{\leq}(f(xs)) \wedge \text{Perm}(xs, f(xs)).$$

It is a refinement of the function value, not a predicate checked independently on a few calls. Under a linear order, uniqueness of a sorted representative of a finite multiset yields idempotence of every function satisfying this specification. Thus $f(f(xs)) = f(xs)$ is a derived behavioral refinement, not necessarily an additional input constraint.

A sorting-by-key interface needs further care. A total preorder on records induced by a key does not uniquely order distinct records with the same key. Stability must be specified separately; sortedness and permutation do not imply it. The type layer should help the author distinguish these contracts, not select the stronger one from the word “sort.”

12.2 Synthesize data and evidence together

The public synthesis target is

$$\{f : A \rightarrow B \mid \text{Behaves}_{P,Q}(f)\}.$$

Operationally, it has two coupled parts: a data hole for f and proof obligations about that same f . The data hole is explicitly delegated under the fixed contract. Every candidate substitution instantiates all dependent obligations transactionally.

A component registry contains both implementations and proved summaries. For insertion sort, an insertion component may have a theorem that preserves sortedness and inserts exactly one additional multiset element. The recursive call sorts the tail under an induction hypothesis; the insertion contract then supplies both clauses of the result specification. The termination argument belongs to the synthesized implementation, not merely to a proof that the intended recurrence would terminate.

This resembles refinement-directed synthesis in Synquid, but the acceptance artifact is a Lean definition and a Lean proof of the selected full contract. Any automated abstraction used to guide search is subordinate to that artifact. A universal property outside the automated fragment remains a Lean proof obligation instead of being silently weakened to a supported length law.

12.3 Partial candidates and the limits of pruning

Suppose a synthesis sketch is $\text{append}(h(xs), xs)$ and the desired output length is $|xs|$. The append length theorem induces the necessary length condition $|h(xs)| = 0$. This constrains the hole before its body is chosen. If the desired result is a permutation of xs , the empty-list solution is compatible. If the desired result contains two copies of every element, it is not.

A solver can operate on a finite abstraction of these constraints. However, three claims must remain distinct:

this completion fails, no completion of this sketch succeeds, no program in the full language succeeds.

A counterexample to one completed candidate proves only the first. To certify exclusion of a whole sketch, one needs a theorem covering every permitted assignment to its holes, or an adequate complete decision procedure for that restricted problem.

Heuristic pruning can still be useful in an explicitly incomplete search mode. It does not threaten the validity of a final kernel-checked result, but it sacrifices completeness. It must not support a claim of nonexistence or an exhaustive-search receipt. Requiring Lean proofs for every search heuristic would needlessly entangle proof soundness with search quality; requiring them for any advertised logical exclusion is the correct boundary.

12.4 Relational specifications and observational summaries

A higher-order component can carry a summary such as

$$(\forall x, y, R(x, y) \rightarrow S(h(x), h(y))) \implies \text{Respectful}_{\text{ListRel}(R), \text{ListRel}(S)}(\text{map } h).$$

Such summaries let a synthesizer compose monotonicity, equivalence preservation, or Lipschitz bounds without unfolding all implementations. The exact relations are parameters, and the theorem behind the summary must remain applicable under their chosen instantiation.

This is also how numerical behavior enters the language. A bound $\|f(x) - f(y)\| \leq L\|x - y\|$ is a universal relational contract; an interval enclosure for one evaluation is an observation. The centered-coupling proof in Section 10 composes the first kind with an input-error observation to obtain an output-error observation. It does not confuse the two kinds of evidence.

12.5 A concrete Leant integration sequence

Leant’s existing carrier-type synthesis remains the first provider, not a component to discard. Its exact rendered candidate, typed provenance where available, and command-local verification context are retained. A Lean-side owner then elaborates the accepted candidate at the fixed refined target and asks for the missing behavioral proof.

The first implementation can use ordinary theorem search and proof tactics for those proofs. A second stage exposes component summaries and residual constraints to an enhanced search interface, so that holes are constrained before enumeration finishes. That second stage is new work: the current Length implementation explicitly does not perform general typed-prefix pruning or counterexample-directed replacement synthesis [14].

Length abstraction is an excellent initial plugin because its intended finite list-spine domain is already sharply delimited. It should remain a declared abstraction, with the exact input/result correspondence retained. Extending it to element multiplicities, orderedness, algebraic invariants, or resource costs requires new semantics and bridges, not just more fields in the same JSON contract.

A useful first obligation population comes directly from the case studies: composing divisibility lemmas to satisfy exact-division guards; assembling local measurability from a phase function and a random variable; combining openness and membership with equality-on-domain; producing a finite reindexing map and its coverage proofs. Measure separately which are solved by one theorem lookup and which require genuine

structural composition. Otherwise improvements from a theorem index may be incorrectly attributed to Leant’s synthesis engine.

12.6 Result taxonomy

The provider protocol should expose independent axes rather than a single “verified” flag.

Receipt or outcome	What it actually authorizes
Carrier elaboration	The exact candidate has the expected coarse Lean type in the recorded context.
Behavioral proof	A retained Lean term establishes the full requested specification of that candidate.
Replayed counterexample	The particular candidate fails the assessed contract at the retained input, under that replay semantics. Publication as a Lean theorem requires a Lean bridge.
Finite-box result	The stated bounded cases were covered; no unrestricted universal conclusion follows.
Search exhaustion	The selected search procedure produced no further candidate within its advertised domain and limits.
Certified restricted absence	A proved negative statement about an explicitly delimited grammar or fragment, with an adequacy theorem if it is transported elsewhere.
Timeout, refusal, malformed reply	An operational outcome; no logical conclusion.

A proof of $\neg \text{Spec}(c)$ refutes candidate c , not the synthesis goal $\exists f, \text{Spec}(f)$. A proof that an abstraction is uninhabited transfers to a Lean target only through a proved implication in the needed direction. Complete search in a restricted logical calculus is not automatically a proof of negation in Lean’s ambient logic.

12.7 Tactic suggestion is a different target

A suggested tactic edit has an origin state and a displayed spelling. Verifying one elaborated variant does not automatically verify a differently printed edit. Acceptance must replay the exact proposed edit from the original state, inspect all resulting goals and metavariables, and distinguish local progress from root theorem completion.

The retained artifact can be the resulting proof term, while the source edit remains a reproducibility and explanation artifact. A stale suggestion after undo or a change to a meaning-bearing instance must not mutate the current state. These transaction requirements are inherited from the syntheses and fit Leant’s deliberate distinction between a callback receipt and stronger retained authority [1, 2, 13].

13 A verified-CAS boundary with an explicit checker argument

13.1 Three execution routes, one specification

A symbolic computation can be accepted through a verified algorithm reduced in the kernel, through an untrusted candidate and a sound certificate checker, or through a generated chain of ordinary theorem applications. These routes may have different costs, but all must establish the same fixed specification at the original Lean expression.

A correctness theorem for an algorithm A has the form

$$\forall i, \text{Spec}(i, A(i)).$$

A byte string claiming to be the output of an external execution is not definitionally $A(i)$. It needs execution evidence or an independent certificate. Similarly,

$$\text{check}(i, o, c) = \text{true} \implies \text{Spec}(i, o)$$

does not suffice without evidence that the check returned true for these exact inputs. This execution distinction is central in both syntheses and in Lean’s validation guidance [1, 2, 18].

13.2 A concrete sparse multivariate certificate format

Fix a variable count k . A monomial exponent is a vector $\alpha \in \mathbb{N}^k$. A computational polynomial is a finite list of pairs (α, c) with $c \in \mathbb{Z}$. A canonical representation sorts exponent vectors lexicographically, combines repeated vectors, and removes zero coefficients.

Before accepting a canonical input, validate the vector dimension, nonnegative integral exponents, integral coefficients, ordering, uniqueness, and absence of zero entries. A noncanonical producer may be normalized by a separate explicitly specified routine. Resource limits on term counts, degrees, coefficient sizes, and multiplication work are operational limits; exceeding them gives a refusal, not a theorem that an identity is false.

For a commutative ring R , a valuation $\rho : \{0, \dots, k-1\} \rightarrow R$, and a list $p = [(\alpha_\ell, c_\ell)]_{\ell=1}^s$, define

$$\llbracket p \rrbracket_\rho = \sum_{\ell=1}^s \iota_R(c_\ell) \prod_{j=0}^{k-1} \rho(j)^{(\alpha_\ell)_j},$$

where $\iota_R : \mathbb{Z} \rightarrow R$ is the canonical homomorphism. This denotation is meaningful even for an unsorted list with repeated exponents.

Addition concatenates and normalizes. Negation negates coefficients. Multiplication forms all pairs of terms, adds their exponent vectors, multiplies their integer coefficients, and normalizes. These are finite executable operations separate from the semantic polynomial type.

Lemma 13.1 (Normalization and arithmetic denotation). *Normalization preserves denotation. The computational operations satisfy*

$$\llbracket p + q \rrbracket_\rho = \llbracket p \rrbracket_\rho + \llbracket q \rrbracket_\rho, \quad \llbracket -p \rrbracket_\rho = -\llbracket p \rrbracket_\rho, \quad \llbracket pq \rrbracket_\rho = \llbracket p \rrbracket_\rho \llbracket q \rrbracket_\rho.$$

Proof. Swapping adjacent terms preserves a finite sum by commutativity. Combining two occurrences of an exponent vector uses $\iota(c)m + \iota(d)m = \iota(c+d)m$. Removing a zero coefficient uses $\iota(0)m = 0$. These operations establish normalization preservation by induction over the chosen sorting and aggregation algorithm.

Concatenation denotes addition of finite sums, and coefficient negation denotes additive inversion. For multiplication, distribute the product of the two finite sums. For each pair of monomials, commutativity and $x^{a+b} = x^a x^b$ identify the product with the monomial whose exponent vector is the componentwise sum. Integer casts preserve the coefficient product. Normalization preservation completes the proof. $\square$

To mechanize this proof, one must choose an actual list normalization algorithm and prove its individual invariants, not cite a conceptual normalization operation. The companion implements a finite dictionary-based model; it is not the proposed Lean implementation of this lemma.

13.3 Ideal-combination checking

For hypotheses $f_1, \dots, f_m$, target g , and supplied multipliers $q_1, \dots, q_m$, the checker first verifies *exactly matching list lengths*. It then computes

$$r = \text{norm} \left(g - \sum_{i=1}^m q_i f_i \right)$$

and accepts precisely when the canonical list for r is empty. A truncating zip is not an acceptable way to ignore extra hypotheses or multipliers.

Theorem 13.2 (Mathematical soundness of the certificate checker). *For valid inputs of common variable arity, if the checker accepts, then in every commutative ring R and under every valuation ρ ,*

$$(\forall i \in \{1, \dots, m\}, \llbracket f_i \rrbracket_\rho = 0) \implies \llbracket g \rrbracket_\rho = 0.$$

Proof. An empty coefficient list denotes zero. By Lemma 13.1, acceptance therefore gives

$$0 = \llbracket g \rrbracket_\rho - \sum_{i=1}^m \llbracket q_i \rrbracket_\rho \llbracket f_i \rrbracket_\rho.$$

Each summand vanishes under the supplied hypotheses, so $\llbracket g \rrbracket_\rho = 0$. Exact arity checking ensures every multiplier is paired with its intended hypothesis. No field property, injectivity of evaluation, or absence of zero divisors is used. $\square$

This theorem is a handwritten proof of the specified algorithm’s mathematical contract, not a Lean-checked proof about the accompanying Python implementation. The first production milestone is its Lean mechanization, with the actual computational normalization and evaluator definitions.

13.4 The original-expression bridge

A reflective tactic must connect that theorem to the user’s expressions. Reification assigns each opaque Lean atom an index and retains a valuation ρ whose entries are those exact expressions, including instantiated types and dictionaries. For every reified input e_i and target e , it constructs

$$r_i : e_i = \llbracket f_i \rrbracket_\rho, \quad r_g : e = \llbracket g \rrbracket_\rho.$$

These bridge equalities can be proved by structural recursion over the reified expression, using the denotation lemmas. An unsupported expression becomes an opaque atom only if its type fits the selected ring interpretation. Two atoms are not identified because their printed names match.

Acceptance then has four visibly separate steps: prove the reification bridges, justify the checker execution, apply Theorem 13.2 to the actual original hypotheses, and transport the conclusion back through r_g . A certificate computed for another valuation, another coefficient ring, or another ordering of hypotheses must not be reused without rebuilding and checking these connections.

13.5 What this checker does not decide

The checker verifies a supplied ideal-combination identity. It is not a complete ideal-membership search procedure. Rejection of one multiplier list proves neither non-membership nor falsity of the target under its hypotheses. An identity for g^n does not prove $g = 0$ in every commutative ring: the nonzero element $2 \in \mathbb{Z}/4\mathbb{Z}$ has square zero.

The denominator example is equally important. X belongs to the ideal generated by $2X$ over $\mathbb{Q}[X]$, but not over $\mathbb{Z}[X]$. Injectivity of the coefficient inclusion does not reflect that ideal membership. A rational-coefficient certificate cannot be imported as an integer certificate without a further theorem explaining the denominators [1, 2].

The round-two reports identify `linear_combination` and Lean’s bounded `grobner` tactic as useful controls. They should be benchmarked in their pinned versions with their actual supported domains, limits, produced proof objects, and failure behavior. The name “Gröbner” is not a license to claim a complete decision procedure for all polynomial goals. This article does not reproduce those tactic measurements or establish a reusable certificate-export API for `grobner`.

13.6 Residual jets fit the same contract discipline

The residual-to-jet theorem from the second-round Locus line of work should be reused, not redescribed as a new FIBER theorem. In a commutative ring R , let $F(Y) \in R[[X]][Y]$, let $F(\Delta) = 0$, and let $\Delta(0) = d$. If the

derivative of the reduced polynomial at d is a unit, then a candidate J with $J(0) = d$ and $F(J) \equiv 0 \pmod{X^N}$ agrees with Δ modulo X^N , for $N \geq 1$ [1, 2].

The proof factors

$$F(J) - F(\Delta) = (J - \Delta)U,$$

where U has unit constant coefficient and hence is invertible as a formal series. Multiplication by U^{-1} gives the desired divisibility by X^N . Root existence, branch agreement, and the unit hypothesis are all separate inputs.

FIBER represents the result as a refinement of the *candidate observation relative to the existing Δ* . It does not replace an arbitrary solution by a canonical construction. Over $\mathbb{Z}/4\mathbb{Z}$, $F(Y) = 2Y$, $\Delta = 0$, and $J = 2X$ show why a merely nonzero reduced derivative is insufficient. For $F(Y) = Y^2 - 1$, choosing $J = -1$ when the retained root is $+1$ shows why the branch condition is indispensable. Those repaired second-round examples become negative typing tests for the wrapper.

14 The author-facing language and its document model

14.1 A small vocabulary, with Lean terms as an escape hatch

The first surface needs only declarations, local bindings, asserted facts, named method applications, witness requests, case splits, and computational requests. Arbitrary Lean terms and proof blocks remain available. A broad natural-language parser is not required to test the design.

A compact grammar skeleton is:

```
statement := theorem name telescope : proposition proof
proof      := by method arguments | begin step* end | lean_block
step       := let name [: type] := term
            | know proposition [by method arguments]
            | obtain pattern : specification [using provider]
            | compute name : specification [using provider]
            | show proposition [by method arguments]
            | cases term with branch+
type       := lean_type | lean_type { binder => proposition }
method     := qualified_contract_name
```

The more readable phrases in the case studies are presentations of registered, namespaced predicates and contracts. They are not a claim that unconstrained English has a unique formal interpretation. A phrase like “continuous on U ” must resolve to a specific predicate, with the ambient field and topology fixed; its expansion is always inspectable.

A *know* line asks for a proof. A *given* binder in a theorem statement declares a hypothesis. A branch assumption is scoped. These operations must not be interchangeable in the editor: automatically turning a failed *know* into a new hypothesis would alter the theorem rather than prove it.

14.2 Three views of one accepted derivation

The *mathematical view* shows objects, choices, substantive conditions, and reasons. The *obligation view* shows all required premises, their status, and where they came from. The *kernel view* exposes the precise Lean declarations and retained proof evidence.

These are views of one typed record, not independently maintained documents. A result displayed as an equality everywhere must actually carry that relation; a domain-restricted equality cannot be rendered with an unqualified equals sign merely because its proof is hidden behind a fold. Branch, precision, domain, completeness, and trust changes remain visible by default, following the second-round visible-obligation boundary [1, 2].

An unresolved premise can remain in a draft, and a conditional theorem may itself be a complete accepted result. What cannot happen is publication of an unconditional conclusion whose guard was only acknowledged as a warning.

14.3 Named reasons constrain rule selection

The command “by reindexing” should select the registered finite-reindexing contract and produce its map, coverage, and summand obligations. An unrelated proof found by global search may be offered as a different plan, but must not silently replace the advertised reason. The same applies to dominated convergence and exact division.

This is an enforceable structural constraint on the derivation record, not a theorem about a reader’s interpretation or the logical necessity of every premise. Determining whether a premise is mathematically indispensable is a stronger question. The FLT proof-path document notes a level-lowering interface whose modularity hypothesis is formally unused because a helper rederives it; this illustrates why a label and a theorem type alone do not establish an exact account of the proof’s reasoning [8].

The initial system should guarantee that the selected contract is applied at the node and that the displayed argument matches its instantiated conclusion and premises. It should expose declared inputs and actual retained dependencies without claiming a minimal explanation. Contract packages still need mathematical review.

14.4 Independently check every assertion

A correct final theorem does not justify an incorrect intermediate sentence. Every asserted node in the document must have its own checked proposition, even if the final proof no longer depends on it after a later edit. Otherwise a theorem prover can certify the last line while the published narrative contains a false unused statement.

Conversely, an author should be able to omit unnecessary intermediate facts entirely. The aim is a faithful mathematical proof, not an exhaustive transcript of search or all failed candidates. Rejected alternatives belong in optional provenance, not the main exposition.

15 Implementation architecture and a realistic first release

15.1 Module responsibilities

Module	Responsibility
<code>Fiber.Core</code>	Reference subtype and behavior encodings, observation records, theorem-backed conversions.
<code>Fiber.Contracts</code>	Checked wrappers, binder roles, interface versions, and exact instantiated requests.
<code>Fiber.Facts</code>	Scope-aware indexing of already proved propositions; no independent logical assumptions.
<code>Fiber.Elab</code>	Bidirectional refinement checking, carrier freezing, dependent obligation assembly, and Lean term generation.
<code>Fiber.Views</code>	Exact presentations and observations, selected maps, same-carrier weakening, and explicit transport.
<code>Fiber.Compute</code>	Certificate data, computational checkers, soundness theorems, execution evidence, and reification.
<code>Fiber.Leant</code>	Candidate/constraint protocol, exact-origin elaboration, behavioral proof requests, and tactic-edit replay.
<code>Fiber.Document</code>	Mathematical rendering, obligation display, statement expansion, and evidence-linked editing.

These are logical ownership boundaries, not a requirement for eight independently deployed services. A first prototype should be one Lean package with a thin optional connection to the existing Leant process.

15.2 Do not duplicate Lean’s workspace

Ordinary local contexts own parameters and hypotheses. Lean environments own declarations. Existing elaborator snapshots own tentative edits. Typeclass mechanisms own selected structures. FIBER adds a searchable fact index, explicit contract instances, refined-hole constraints, and source-to-evidence links only where those enable operations not already exposed conveniently.

The index should be reconstructible from a snapshot’s local declarations and retained accepted nodes. It should not become an independently mutable store of purported truths. An accepted fact is immutable evidence; changing its source or dependencies creates a new fact identity or a new proof of the same proposition.

15.3 Elaboration pipeline

1. Elaborate and freeze the outer statement, structures, conventions, and explicit construction specifications. Surface unresolved meaning-bearing choices.
2. Elaborate the next mathematical step against an expected contract. Produce carrier terms, proof skeletons, and an ordered dependent obligation graph.
3. Resolve inexpensive property obligations from local evidence and approved conversion rules. Do not run expensive global synthesis inside every coercion attempt.
4. Send remaining explicit requests to theorem search, Leant, or a computation provider with a whole-request budget and immutable origin.
5. Replay returned evidence or candidate edits in a child elaborator state. Reify and check computational results at the exact original target.
6. Commit only a fully consistent transaction. Export the accepted Lean proof objects and a document record with matching guarantees.

The most important engineering boundary is between stages 1 and 2: proof search may solve evidence holes, and explicitly delegated data holes, but may not choose a new theorem statement.

15.4 A staged prototype that tests the new idea

Milestone 1: same-carrier refinements. Implement the expected-refinement judgment, ordinary introduction and weakening, scoped fact lookup, and statement freezing. Use basic nonzeroness, interval membership, and exact integer quotients. Export ordinary Lean. No external solver is necessary.

Milestone 2: the non-vacuous Frey transport. Implement the exact-quotient and base-change packages under (F_0) , including the three missing-premise neighbors. The measurable deliverable is the complete transport theorem with generated and inspectable guards, plus a compatibility adapter to the existing definitions. Give every wrapper to the baseline Lean proof.

Milestone 3: one sound computational checker. Mechanize the sparse-polynomial representation, normalization, denotation lemmas, checker theorem, and reification bridge. Test genuine multivariate cases, zero divisors, large coefficients, arity errors, and corrupted certificates. The Python model shipped here is preparation for this milestone, not completion of it.

Milestone 4: analytic scopes. Implement the phase-function and dominated-convergence contracts, then the local-derivative wrapper. This determines whether scope-sensitive inference really generalizes beyond algebra and whether measure- or range-specific refinements stay manageable.

Milestone 5: refinement-directed Leant. Start with candidate plus universal proof requests. Then introduce one backward-constraint domain with a formally stated abstraction and an exact candidate-origin bridge. Compare genuine compositions against single-lemma lookup. Do not rewrite the Haskell search engines in Lean merely to fit the architecture.

Milestone 6: a surface only after evidence. Add the compact document syntax once the earlier stages have established stable contracts. A renderer over ordinary Lean remains a valid endpoint if a separate authoring language offers no measured improvement.

15.5 Performance hazards worth addressing early

Subtype wrapping may increase projection and coercion noise; keep the unbundled local representation and only package at API boundaries. Repeated proof search at implicit arguments may explode; index by exact requested predicate shape and maintain explicit budgets. Reflexive checking can spend more time normalizing certificate data than searching for it; choose representations for the strict checker, not merely for mathematical elegance.

A property index can also be too eager. A large conjunction of irrelevant refinements may hurt inference more than it helps. Preserve summaries as separate entries and expose only demanded projections. Term sharing should reduce repeated certificate or proof subterms without replacing semantic identities by unverified hash equality.

No native-versus-kernel crossover number is supplied here. It depends on the actual checker, coefficient representation, proof reconstruction, and pinned toolchain. The historical second-round experiment does not determine it for this design. Benchmark all three routes on the intended workloads before selecting an accelerated profile.

16 Trust, replay, and migration

16.1 Logical trust is not an execution-status flag

A published mathematical result carries an actual Lean proposition and proof. A policy checker inspects the transitive dependencies of that proof under the pinned environment. It does not accept an untrusted provider's statement that the proof is "strict" or "kernel checked." Lean's proof-validation documentation explains why independent validation and awareness of native-evaluation trust are relevant here [18].

The source statement and the proof's axiom inventory serve different purposes. A proposition may typecheck while depending on assumptions the project does not permit. Conversely, a native process may safely propose a certificate if the certificate is subsequently checked through the strict path. Producer execution need not be trusted simply because it is fast.

16.2 Operational capabilities should not become logical assumptions

A request may permit network access, a native solver, a time budget, or a particular certificate format. These are operational permissions. They are not premises in the mathematical theorem.

Likewise, a "pending" or "accepted" state belongs to the transaction manager. A dependent record containing a proof of P can represent established evidence; a JSON tag named `proved` cannot. Logical facts are reusable and need no linear type discipline. A transaction ticket may be single-use to prevent duplicate commits, but that is a property of the editing protocol, not a restriction on using a theorem twice.

For verified programs with state or resource bounds, a genuine effectful specification can be appropriate: a transition relation, trace semantics, or costed evaluator can be part of the mathematical target. A timing observed

in one run is not a proof of a worst-case bound. This keeps the useful lesson from Dijkstra-style contracts without forcing every proof-search event into the object logic.

16.3 Retained evidence and checked reuse

A retained request includes the environment identity, exact telescope, fixed input expressions, selected structures, requested relation, output, and proof or certificate. Source text and search history are companions. They are not the sole evidence if later elaboration might choose different defaults.

Reuse across a new request is a theorem application. A higher-precision jet may answer a lower-precision request; equality on a larger domain may answer equality on a subset; a narrower enclosure may answer a wider one. These are separate entailments, not a single numerical “strength” score. The current context must prove the old guards or a bridge to the new specification, and the accepted axiom policy must still be satisfied [1, 2].

16.4 Toolchain upgrades and collaboration

A toolchain upgrade requires checking in the destination environment and issuing a new receipt. A renamed native-evaluation axiom is not made acceptable by a textual allowlist substitution. The meanings and dependencies of the declarations, as well as the destination policy, must be re-evaluated.

For collaboration, merge immutable accepted nodes by their mathematical dependencies. If two authors select different witnesses, representation maps, or structure dictionaries for one source object, the merge is a semantic conflict even when line-based merging succeeds. Two proofs of the same proposition need not be a conflict. Two same-carrier refinement paths can use Proposition 5.1; arbitrary representation routes cannot.

17 Evaluation: isolate the effect of refinements

17.1 An ablation that charges infrastructure fairly

A nested experiment, adapted from the syntheses’ stronger-baseline requirement, should use these treatments:

Arm	Available facilities
L0	Idiomatic Lean with the original source library and ordinary tactics.
L1	L0 plus all FIBER theorem wrappers, definition packages, and certified computational interfaces.
L2	L1 plus the same theorem search, Leant providers, and retained computational results used by FIBER.
L3	L2 plus refinement-directed elaboration accessible inside Lean syntax.
F	L3 plus the compact source and mathematical document interface.

L1 minus L0 measures library engineering. L2 minus L1 measures added search services. L3 minus L2 is the main test of this proposal’s source-type extension. F minus L3 tests the additional language and presentation. A rendering-only condition is useful for a separate reading experiment. These are not claims that the effects are additive or independent; the design exposes where interactions must be measured.

Report infrastructure construction and maintenance, per-task adapter work, proof-writing effort, and observed reuse separately. Do not hide dozens of task-specific theorems behind a one-line method and count only that line. Equally, do not charge a genuinely reusable theorem anew for every application without reporting its reuse.

17.2 Tasks and leakage

Use families spanning exact arithmetic, finite reindexing, local analysis, measure-theoretic convergence, and behavioral synthesis. The inspected Frey, binomial, autonomous, and spline files are development material. Seal evaluation families separately, excluding aliases and easy downstream consequences of training targets.

For contradiction-heavy developments, exclude final contradiction theorems and their descendants from proof-search components when measuring the intended method. The weakened Frey arithmetic (F_0) gives a particularly clean control: its parameter space is inhabited, so success cannot be attributed to the impossibility of a full counterexample package.

Randomize task order, counterbalance interface exposure, and report experience with Lean. Use paired comparisons within task families rather than a single aggregate source-line count. Report uncertainty across tasks and users; do not treat many near-duplicate theorems as independent evidence of generality.

17.3 What to measure

The important quantities include accepted theorem strength, author time, meaningful choices requested, unresolved-guard quality, and repair effort after a controlled library change. Also record elaboration/search/checking time separately, certificate and proof size, memory, and the proportion of obligations solved by direct lookup versus multi-step composition.

Measure erroneous *interpretations* as well as erroneous proof attempts. In a blind reading test, ask readers to reconstruct the object, domain, branch, quantifier dependencies, precision, and completeness of a compact result. Pair plausible positive and negative neighbors. Then reveal the expanded obligation view and measure whether it corrects misunderstandings, following the strongest usability proposal in the syntheses [1, 2].

Avoid universal success claims based only on an adversarial suite. A system that rejects every request can pass all negative cases. Every negative mutation needs a positive neighbor, and refusal, timeout, proof failure, and actual counterexample must be distinguished.

17.4 A targeted mutation register

The following table is an implementation test specification. “Finite” identifies a mathematical neighbor exercised by the Python companion; it does not mean the FIBER frontend behavior was executed.

ID	Mutation	Required frontend behavior	Here
T1	Remove oddness from (F_0).	Leave the divisibility guard open; show the $p = 4$ neighbor.	Finite
T2	Remove evenness of b .	Reject the claimed exact quotient, not integer division itself.	Finite
T3	Permit $p = 3$ for the second quotient.	Require divisibility by sixteen; do not round.	Finite
T4	Cast $3/2$ from integer to rational division.	Require exact division evidence or preserve the integer quotient.	Finite
T5	Replace germ equality by equality at a point.	Refuse derivative transfer through that contract.	Finite
T6	Change the measure of a domination fact.	Require a measure-transport theorem.	Spec.
T7	Infer a common conull set for uncountably many parameters.	Require a separate uniform theorem.	Spec.
T8	Use length preservation as a sorting proof.	Keep permutation and order obligations open.	Finite
T9	Use set equality instead of multiplicity preservation.	Reject the weaker specification.	Finite
T10	Use a finite sample bank as a universal proof.	Retain bounded evidence only.	Finite
T11	Refute one completion, then reject every sketch completion.	Require a universally scoped exclusion or label a heuristic.	Spec.
T12	Truncate mismatched certificate lists.	Reject malformed arity before checking.	Finite

ID	Mutation	Required frontend behavior	Here
T13	Corrupt a polynomial coefficient.	Reject the certificate at its actual target.	Finite
T14	Eliminate a square in a ring with nilpotents.	Require the appropriate reducedness or regularity premise.	Finite
T15	Invert a merely nonzero or regular integer.	Require a unit in the chosen ring.	Finite
T16	Move a branch-local fact into its sibling.	Reject the context embedding or retain its guard.	Spec.
T17	Change a meaning-bearing instance after a cached proof request.	Re-elaborate and invalidate the old origin receipt.	Spec.
T18	Modify the displayed tactic after probing it.	Replay the exact displayed edit from its origin.	Spec.
T19	Hide an unused false intermediate assertion.	Reject that document node independently.	Spec.
T20	Promote a finite jet to an analytic identity.	Require both an exact-series argument and realization hypotheses as applicable.	Spec.
T21	Count a witness plus coverage as complete solving.	Demand soundness for every listed solution.	Finite
T22	Reject equality from a collapsed interval merely because it is an enclosure.	Accept the equal-endpoint bridge.	Finite
T23	Reuse a certificate at a different opaque-atom valuation.	Rebuild and prove the original-expression bridge.	Spec.
T24	Import the final FLT contradiction into the arithmetic benchmark.	Mark the run contaminated; use (F_0) .	Spec.

17.5 Stopping and revision criteria

Keep the refinement layer only if it improves a useful combination of authoring, diagnostics, semantic reading, or maintenance over L2. If L1 explains most gains, retain the definition and theorem packages. If L3 improves authoring but F does not, retain Lean syntax with refinements and the optional renderer. If automatic property closure becomes unpredictable, narrow the rule set and expose explicit requests rather than broadening the trusted machinery.

The design’s central hypothesis is testable: scoped, compositional properties can reduce repeated proof assembly without concealing changes of meaning. A failure of that hypothesis on the chosen implementations is information about the implementation and abstraction boundary, not a reason to weaken the mathematical acceptance condition.

18 Answers to the third-round engineering questions

The revised syntheses ask overlapping questions. The following decisions are specific enough to guide implementation, while distinguishing questions that still require measurements [1, 2].

Checker soundness and the first package. Start with the exact-quotient interface and non-vacuous scalar-extension theorem for the refinement frontend. In parallel, mechanize the sparse certificate checker of Section 13, including its actual normalization and reification. This does not claim the Python model completes the reports’ requested Lean checker milestone.

Bounded algebra and execution trust. Treat existing bounded polynomial tactics as proof providers with pinned contracts, not as a complete CAS. Keep kernel reduction and reconstructed proofs as the strict routes. The threshold for native evaluation is an experimental result to obtain on the chosen checker data; it is not inferred from the historical binomial-factor timing family.

Workspace and partiality. Use Lean’s context and snapshots as the primary state. Add only evidence indexing and contract/obligation records. Preserve existing total operations when importing Lean. Offer explicitly domain-indexed partial objects when the mathematics benefits from them, without confusing their solution sets with totalized equations.

Leant’s concrete contribution. Measure compositions of divisibility, locality, measurability, and finite-map obligations. Introduce behavior proofs after candidate elaboration before attempting a general constraint-directed engine. Current replay filtering remains a useful bounded negative-evidence stage, not universal verification.

Witness delegation and display. Delegate witnesses only under a frozen, explicit specification. Retain the accepted object and its dependencies; expose noncanonical choices. Show domains, branches, precision, complete-versus-partial solution claims, and unresolved guards by default. Distinguish all of these from optional search history.

Migration and collaboration. Revalidate proof dependencies and policy in the destination toolchain. Merge proofs of the same proposition freely when appropriate; treat conflicting data choices, maps, and instances as semantic conflicts. Do not treat a cosmetic source merge or axiom rename as validation.

Mutation execution and campaign design. The 24-row register here is smaller than the syntheses’ consolidated backlog because it focuses on the type extension’s failure modes. Its source-neighbor arithmetic is partially exercised; frontend enforcement remains to be implemented. The next useful artifact is a Lean package for these contracts and tests, not another untested surface spelling of the same acceptance boundary.

19 Conclusion

Lean’s logical language is already rich enough to express the properties and behaviors mathematicians use. The opportunity is to make those properties an active, compositional part of elaboration: information should follow the same object through its uses, and expected behavior should constrain constructions before they are complete.

FIBER’s recommended extension is consequently conservative at the kernel and ambitious at the elaborator. It combines same-carrier refinement inference, explicitly indexed analytic and computational guarantees, theorem-backed transport, and coupled synthesis of values with their specifications. The Frey exact-quotient case explains how definitions can remove repeated transport work without erasing divisibility. The probability case explains why measure, locality, and uniformity must be first-class indices rather than informal adjectives. The synthesis case explains why length filtering is useful but not the endpoint.

The work that remains is substantial and specific: a real refinement elaborator, Lean proofs for the computational checker and its reification, a narrow contract-directed synthesis interface, and a fair comparison with Lean supplied with all the same libraries. No trusted CAS conversion rule is needed to begin. The guiding principle is to let the author state the mathematical information once, let the system carry and specialize its evidence, and make every change of meaning an explicit theorem or an explicit choice.

A Reference encoding and artifact status

The archive includes `CoreEncoding.lean`, a small ordinary-Lean reference encoding of refinements, behavior composition, and anchored observations. It contains no FIBER parser and no polynomial checker. It was not compiler-checked in this authoring environment, which had no Lean executable available. Its purpose is to show the proposed semantic boundary in concrete terms, not to claim a completed Lean prototype.

The essential representation is:

```
abbrev Refined (A : Type u) (P : A -> Prop) := { x : A // P x }

def weaken (h : forall x, P x -> Q x) (x : Refined A P)
  : Refined A Q :=
  <x.val, h x.val x.property>

def Behavior (f : A -> B) (P : A -> Prop) (Q : A -> B -> Prop) :=
  forall x, P x -> Q x (f x)
```

This display uses ASCII presentation notation for binders and pairs; the companion file uses ordinary Lean notation and supplies the full universe and implicit-argument declarations. The generic certificate-bridge theorem in that file *takes checker soundness and execution evidence as arguments*. It is not the missing polynomial soundness proof in disguise.

Artifact or claim	Status in this delivery
Both revised synthesis reports and selected repository sources	Read through their pinned GitHub sources; scope and locators recorded.
Core refinement calculus and elaboration argument	Specified here; handwritten relative soundness proof.
Exact quotient and centered coupling statements	Mathematical proofs in the article; finite neighbors exercised.
Sparse certificate checker contract	Algorithm specified and mathematical soundness proved; Lean mechanization not supplied.
Python polynomial model and example suite	Executed: 1,298 finite checks in 31 groups.
<code>CoreEncoding.lean</code>	Illustrative ordinary-Lean source; not compiler-checked here.
FIBER frontend, source renderer, Leant refinement-directed extension	Proposed, not implemented.
Complete ProvelT or FLT repository validation	Not performed.
Performance and usability comparison	Designed, not measured.

B Reproducing the finite checks

Run

```
python tools/check_examples.py
```

from the extracted archive. The program uses only the Python standard library and targets Python 3.9 or newer. The recorded execution used Python 3.13.5 and seed 20260906. It writes `evidence/checks.json` with the exact group counts and operational limits.

The tests include 60 generated formal ideal-combination identities and 60 corrupted neighbors, their integer,

modulo-four, and dual-number evaluations, malformed arity and polynomial inputs, 288 exact-division parameter triples and their rational casts, and 381 endpoint or midpoint instances of the centered-tail inequality. Other groups are small positive or negative examples distinguishing specification strengths. These counts are not independent theorem discoveries or a statistical confidence level.

The code uses exact integers and fractions. Its dual-number evaluations use pairs (a, b) for $a + b\epsilon$, with $\epsilon^2 = 0$, so a field-only assumption cannot be hidden by testing solely over rationals. The mathematical checker theorem establishes soundness over arbitrary commutative rings; these evaluations only exercise the implementation on selected examples.

The program is a bounded executable model, not a hardened adversarial certificate service. A production implementation must bound serialized input sizes and parsing as well as arithmetic work, and use a formally connected normalization implementation. A Python exception is classified as an operational refusal, not an algebraic refutation. There are no kernel receipts in `checks.json`.

To rebuild the article, run `build.sh`, which invokes pdfLaTeX repeatedly to settle cross-references. The source is self-contained apart from ordinary TeX packages; no bibliography database or external image assets are required.

C Source identities and review map

The following commits identify the review snapshots. Long identities are also stored in `sources.json`, along with the inspected paths. Git blob identifiers are recorded where returned by the connector; they are Git object identities, not invented SHA-256 checksums of downloaded files.

Repository	Commit
VladimirReshetnikov/Brainstorm	b052ec011beb96a7c48df2a9988823f0f779b445
VladimirReshetnikov/ProveIt	24ce8bd743eaab64a91ce90725ea00f498d319d2
VladimirReshetnikov/Leant	3a40904be8a410d832d9ab6900b3d3e7b425ecb
anthropics/fermats-last-theorem	aa2d8b34692b16c70f699536de0d8e75b9a3e9ef

The Brainstorm synthesis sources were read across contiguous portions of their TeX files, including their conclusions and question registers. The article relies on their consolidated accounts of the nine individual proposals; it does not claim independent line-by-line review of all nine proposals or execution of their companions.

The ProveIt review covers the binomial inversion proof through its generic sequence interface, the autonomous-derivative file, and the spline weak-convergence file. The FLT review covers its README, proof-path document, Frey package definition, the map-proof file and its arithmetic helpers, and selected portions of the unusually large no-Frey-package module, including its final solution through source search. Leant review is focused on the README, the engine boundary, verification module, and the current Length documentation. These are representative-source reviews, not whole-repository code audits.

The supplementary public references below were consulted on 6 September 2026. Moving latest documentation is used for conceptual background, not as a substitute for the repository toolchain pins. The source register makes this distinction explicit.

References

- [1] Codex. *Mathematical Objects, Certified Computation: A Unified Evaluation of Nine Proof-Language Proposals*. Revised round-two synthesis, Brainstorm, 2026.
<https://github.com/VladimirReshetnikov/Brainstorm/blob/b052ec011beb96a7c48df2a9988823f0f779b445/docs/round-2/synthesis/unified-report.tex>.

- [2] Claude (Anthropic). *Nine Workbenches: Certified Computation in a Mathematician’s Proof Language over Lean*. Revised round-two synthesis, Brainstorm, 2026; source includes the accepted parallel-review corrections and third-round questions.
https://github.com/VladimirReshetnikov/Brainstorm/blob/b052ec011beb96a7c48df2a9988823f0f779b445/docs/round-2/unified_report/unified_report.tex.
- [3] V. Reshetnikov and contributors. ProveIt, *lean-toolchain*.
<https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/lean-toolchain>.
- [4] V. Reshetnikov and contributors. ProveIt, *BinomialInversion.lean*.
<https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/BinomialInversion.lean>.
- [5] V. Reshetnikov and contributors. ProveIt, *AutonomousIteratedDeriv.lean*.
<https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean>.
- [6] V. Reshetnikov and contributors. ProveIt, *UniformSplineWeakConvergence.lean*.
<https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/UniformSplineWeakConvergence.lean>.
- [7] Anthropic and contributors. *Fermat’s Last Theorem*, repository README and validation description.
<https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/README.md>.
- [8] Anthropic and contributors. *The route of the proof*, PROOF-PATH.md.
<https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/PROOF-PATH.md>.
- [9] Anthropic FLT repository contributors, with source material adapted from the Imperial College London FLT project. *Def_FLTPrelim_FreyPackage.lean*. The source credits the Imperial College London FLT formalization.
https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/Definitions/Def_FLTPrelim_FreyPackage.lean.
- [10] Anthropic FLT repository contributors. *S_FreyPackage_freyCurveInt_map.lean*.
https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S_FreyPackage_freyCurveInt_map.lean.
- [11] Anthropic FLT repository contributors. *S_FreyPackage_no_frey_package.lean*. Selected context-management declarations and final solution reviewed.
https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S_FreyPackage_no_frey_package.lean.
- [12] V. Reshetnikov and contributors. Leant, *Engine.hs*, engine boundary and typed candidate provenance.
<https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Engine.hs>.
- [13] V. Reshetnikov and contributors. Leant, *Verification.hs*.
<https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Verification.hs>.
- [14] V. Reshetnikov and contributors. Leant, *Length behavioral ranking and replay-authorized filtering*.
<https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/docs/length-ranking.md>.
- [15] Lean team. *The Type System*. Lean Language Reference.
<https://lean-lang.org/doc/reference/latest/The-Type-System/>.

- [16] Lean team. *Subtypes*. Lean Language Reference.
<https://lean-lang.org/doc/reference/latest/Basic-Types/Subtypes/>.
- [17] Lean team. *Coercions*. Lean Language Reference.
<https://lean-lang.org/doc/reference/latest/Coercions/>.
- [18] Lean team. *Validating a Lean Proof*. Lean Language Reference.
<https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
- [19] N. Polikarpova, I. Kuraj, A. Solar-Lezama. *Program Synthesis from Polymorphic Refinement Types*. PLDI 2016, pp. 522–538; extended version arXiv:1510.08419.
<https://arxiv.org/abs/1510.08419>.
- [20] A. Solar-Lezama, with materials developed in collaboration with N. Polikarpova. *Lecture 22: Synthesis with Refinement Types*. Introduction to Program Synthesis, 2018.
<https://people.csail.mit.edu/asolar/SynthesisCourse/Lecture22.htm>.
- [21] LiquidHaskell contributors. *Papers and refinement-reflection resources*. Official project documentation, including the POPL 2018 refinement-reflection work.
<https://ucsd-progsys.github.io/liquidhaskell/papers/>.
- [22] F* team. *Proof-Oriented Programming in F*: Getting off the ground*.
https://fstar-lang.org/tutorial/book/part1/part1_getting_off_the_ground.html.
- [23] F* research team. *Dijkstra Monads for Free*. Official paper page and abstract.
<https://fstar-lang.org/papers/dm4free/>.
- [24] S. Tobin-Hochstadt, V. St-Amour, E. Dobson, A. Takikawa. *Occurrence Typing*. The Typed Racket Guide.
<https://docs.racket-lang.org/ts-guide/occurrence-typing.html>.